TECHNICAL PROJECT DOCUMENT

Super Admin Portal

Features, technology stack, architecture, security, setup, and complete deployment guide

Purpose: Reference blueprint for building a related project
Repository reviewed: Full frontend and backend codebase
Prepared: 27 August 2026
Confidentiality: No live secrets or credentials included

Executive summary

The Super Admin Portal is a multi-role enterprise operations platform. A single React application presents specialized workspaces for super administrators, administrators, executives, HR, managers, employees, finance, IT, law, media, sales, and freelancers. A modular Express API provides authentication, authorization, business workflows, reporting, file handling, real-time chat/notifications, caching, auditability, and cross-project integrations.

Core value: one secured control plane for people, departments, projects, approvals, reporting, communications, and connected products, while keeping credentials and upstream service calls on the backend.

1. Product scope and feature catalogue

Role and access model

The backend defines a hierarchy and explicit permissions. Roles include `super_admin`, `admin`, `ceo`, `hr`, IT manager/admin/employee/HR, finance manager/employee, media head/sales/marketing, law head/employee, and `freelancer`. UI route guards improve navigation, but backend middleware remains the authority.

Portal / audience	Main capabilities
Super Admin	Global dashboard; company controls; portal access; feature flags; system health; users, departments and settings; logs, security and audit visibility; portfolio and project oversight.
Admin	User CRUD and export; role/permission management; department overview; workflows; reports and analytics; sales submissions/questions; projects; legal registry; support center; outsourcing administration.
CEO	Executive KPIs; department statistics; employee and project updates; revenue/product/media/sales analytics; reports; legal approvals; notifications and chat.
HR	Employee directory and records; recruitment/applicants/job posts; attendance; leave balances, policy and approvals; performance/appraisal; work reports; complaints, notices, holidays; tasks, communication and outsourcing tracking.
Manager	Operations dashboard; project and team management; work approval board; tasks; leave approval; reports and exports; recruitment, products, outsourcing and team chat.
Employee	Personal dashboard; assigned projects/tasks; attendance; job browsing; leave requests; employee documents; team directory, profile and chat.
Finance	Accounts, budgets, cost centers, vendors, clients, expenses, invoices/notes, payments, payroll, journal entries, compliance, approval workflows, audit logs and financial reports.
IT	IT dashboard; assets and tickets; infrastructure/support/security workflows; department settings, support, projects and reporting.
Law	Contracts; legal documents and version history; compliance/regulatory tracking; litigation/cases; IP; risk and advisory; policy review/approval; reporting and audit support.
Media & Sales	Media projects and workspace; campaigns, assets, deadlines and approvals; marketing plans; sales questionnaires/submissions; revenue and team analytics.
Freelancer / Outsourcing	Dashboard, jobs, contracts, milestones, payments, work sessions/time logs, profile, support, project cards, EdifyEight teacher/material administration, and EFNBMMS administration.

Shared platform features

Identity & governance

- JWT access and refresh tokens
- Sessions, password flows and external auth
- Role hierarchy and granular permissions
- Project/workspace access enforcement
- Audit and activity logs

Collaboration

- Real-time chat over Socket.IO
- Notifications and dashboard events
- Support ticket flows
- Tasks, workflows and approvals
- Role-specific communication views

Data & insight

- Role-specific dashboards and KPI cards
- Charts, filters, pagination and exports
- Reports and department statistics
- Cache policies and invalidation
- Structured operational logging

User experience

- Responsive portal layouts
- Dark/light theme and reusable UI kit
- Offline banner and local cache support
- Loading, empty and error states
- Optimistic mutations and query prefetching

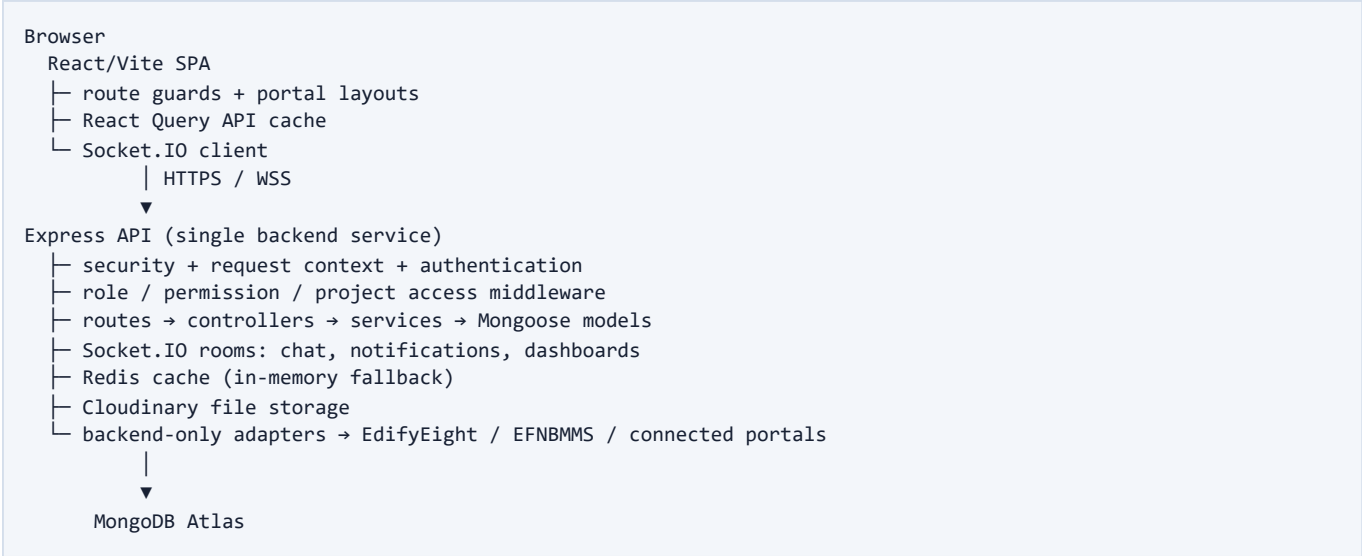
Project workspace

The backend is the authoritative catalogue for HOUSE OF MUSA products: MATEBID, THE BETTER PASS, EDIFYEIGHT, YARROWTECH (EEC-B2B, EFNBMMS, ESPORTSM, ERMS, EHC, SMARTFARMING), HIREME, ARTBLOCK, and GREENBAR. Shared workspace APIs return both the hierarchy and the authenticated user's access envelope.

2. Technology stack

Layer	Technology	Responsibility
Frontend	React 19, React DOM, React Router 7	Single-page application, portal routing, protected/role routes.
Build & styling	Vite 7, Tailwind CSS 4, PostCSS, Autoprefixer	Fast development/build pipeline and responsive styling.
Server state	TanStack React Query 5	Fetching, cache, retry, invalidation, pagination and optimistic updates.
Visualization	Recharts 3; Three.js, React Three Fiber/Drei	Business charts and optional 3D/interactive visuals.
Backend	Node.js 18.18+, Express 4	REST API, middleware chain, domain modules and integrations.
Database	MongoDB with Mongoose 8	Users, roles, operations, finance, HR, legal, outsourcing and system data.
Real time	Socket.IO 4	Chat, notifications and live dashboard updates.
Cache	Redis 6 client; in-memory fallback	Dashboard/list/reference caches, locks, metrics and invalidation.
Files	Multer, Cloudinary	Upload validation and hosted document/media storage.
Security	JWT, bcryptjs, Helmet, rate limiting, mongo sanitize, validators, CORS	Authentication, password hashing, secure headers, abuse/input protection.
Logging/jobs	Pino/Pino HTTP, node-cron	Structured logs, request context, scheduled legal/operational tasks.
Testing/docs	Node test runner, Swagger output, Postman collection, checklist scripts	Infrastructure/access tests, API exploration and role verification.
Hosting	Render, Vercel, MongoDB Atlas, optional Redis/Cloudinary	API hosting, static SPA hosting and managed production services.

3. Architecture, data, and integrations



Backend organization

- `routes/` exposes feature endpoints and attaches middleware.
- `controllers/` translates HTTP input/output and delegates work.
- `services/` contains business rules, workflows, exports, proxying and notifications.
- `models/` contains Mongoose schemas grouped by domain.
- `modules/` encapsulates richer domains such as finance, media, law, IT and chat.
- `middlewares/` handles auth, roles, permissions, validation, uploads, audit, cache headers/invalidation and errors.
- `infrastructure/cache/`, `logger/`, `sockets/` and `jobs/` provide cross-cutting runtime services.

Important data domains

Auth and governance; organization and departments; common projects/tasks/messages/notifications; HR and performance; employees; managers; finance; legal; IT; media/sales; outsourcing; portfolio/support; super-admin control and system health.

EdifyEight gateway pattern

The browser calls only this portal's API using the user JWT. The backend checks workspace/feature permissions and calls EdifyEight's protected internal teacher and study-material APIs with a private service token. This prevents service credentials and internal endpoints from reaching the browser. EFNBMMS uses a separate adapter and route.

> **Reusable integration rule:** catalogue entry → backend adapter/proxy → authorization middleware → audit logging → upstream error mapping → frontend consumes only the local API.

4. Security and quality attributes

Concern	Current design	Production recommendation
Authentication	JWT access/refresh configuration, sessions, password hashing.	Use long random secrets, rotate them, enforce HTTPS, short access-token TTL and revocable refresh sessions.
Authorization	Roles, hierarchy, permissions, workspace/project middleware.	Test every write endpoint for deny-by-default behavior; remove temporary freelancer bypasses.
Input/API	Validators, Mongo sanitization, upload middleware, rate limits.	Add size/type limits per endpoint and centralized validation schemas.
Browser boundary	CORS allow-list, Helmet, backend-only upstream credentials.	Set exact production origins; never use wildcard CORS with credentials.
Auditability	Audit/activity/system logs and request context using Pino.	Centralize logs, define retention, alert on auth failures and privileged actions.
Availability	Health endpoint, compression, cache fallback, optional warm ping.	Use managed Redis for multiple instances; monitor DB/cache/upstream health.
Secrets	Environment-based configuration and example templates.	Keep real `.env` files out of Git; use host secret stores and rotate anything previously exposed.

> **Security note:** this guide intentionally lists variable names only. Never paste live MongoDB credentials, JWT secrets, Cloudinary secrets, or integration tokens into documentation or frontend variables.

5. Local setup and configuration

Prerequisites

- Node.js 18.18 or newer and npm
- MongoDB locally or an Atlas connection
- Optional Redis and Cloudinary accounts
- Connected-project URLs/tokens only when those integrations are tested

Backend

```
cd backend
copy .env.example .env
npm install
npm run dev

# Verify
# http://localhost:5000/health
```

Frontend

```
cd frontend
copy .env.example .env
npm install
npm run dev

# Default Vite URL: http://localhost:5173
```

Configuration inventory

Group	Variables	Required?
Core	`NODE_ENV`, `PORT`, `MONGO_URI`, `CORS_ORIGIN`	Mongo URI and production CORS are required.
Authentication	`JWT_SECRET`, `JWT_REFRESH_SECRET`, expiry values	Secrets required in production.
Cache	`CACHE_ENABLED`, `REDIS_URL`, timeouts, locks and TTL variables	Redis optional for one instance; recommended for scale-out.
Files	`CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET`	Required for real hosted uploads.
Integrations	Portal URLs, shared secret, EFNBMMS URL/token, EdifyEight URL/teacher URL/material URL/token	Required only for enabled integrations.
Operations	`LOG_LEVEL`, registration switch, Render keep-alive settings	Optional/tunable.
Frontend	`VITE_API_URL`, optional outsourcing URL, `VITE_LOG_LEVEL`	API URL required in deployment.

Only variables prefixed with `VITE_` are embedded into the browser build. They must never contain secrets.

6. Production deployment

Recommended topology

- **MongoDB Atlas:** managed database.
- **Render Web Service:** backend from `backend/`.
- **Vercel:** frontend static build from `frontend/`.
- **Optional managed Redis:** shared cache for multiple backend instances.
- **Cloudinary:** persistent uploaded assets.

Step 1 — database

1. Create an Atlas project, cluster, database user and least-privilege network rule.
2. Obtain the hosted connection string and set `MONGO_URI`.
3. If migrating local data, use `mongodump` and `mongorestore`; validate counts before switching traffic.
4. Create indexes and back up the database before production use.

Step 2 — backend on Render

The checked-in `render.yaml` defines a Node web service rooted at `backend`, using `npm install`, `npm start`, and health path `/health`.

```
Root directory: backend
Build command:  npm install
Start command:  npm start
Health check:   /health
```

Set production environment variables in Render, not in Git. Minimum: `NODE_ENV=production`, hosted `MONGO_URI`, frontend `CORS_ORIGIN`, unique JWT secrets, and required integration credentials. Add Cloudinary and Redis configuration when used.

Step 3 — frontend on Vercel

```
Root directory:   frontend
Build command:    npm run build
Output directory: dist
VITE_API_URL:     https://your-backend-service.onrender.com
```

The checked-in `vercel.json` rewrites all SPA paths to `index.html`, allowing direct links such as `/outsourcing/edifyeight`.

Step 4 — final CORS wiring

1. After Vercel provides the public frontend URL, set it as the backend's exact `CORS_ORIGIN`.
2. For several approved domains, use the comma-separated format supported by the backend.
3. Redeploy/restart the backend.
4. Test both HTTP requests and Socket.IO connections from the production domain.

Step 5 — connected services

Replace every localhost upstream URL with a deployed HTTPS endpoint. Ensure EdifyEight's backend has the matching service token. Keep the token only in both servers' secret stores. Configure EFNBMMS independently.

7. Verification, operations, troubleshooting

Release acceptance checklist

- □ Backend `/health` returns success and database state `connected`.
- □ Frontend loads from a direct nested route and refreshes without 404.
- □ Login, refresh token, logout and denied-role behavior work.
- □ Super Admin/Admin/CEO/HR/Manager/Employee/department/freelancer smoke paths work.
- □ User/role changes and privileged writes create appropriate audit records.
- □ Chat and notifications connect over production WebSocket transport.
- □ Upload/open/delete works using hosted storage.
- □ EdifyEight teacher/material CRUD and EFNBMMS calls work without exposing service tokens.
- □ Reports/exports, pagination, search and dashboard data work.
- □ Logs contain request context but no passwords, tokens or sensitive payloads.
- □ Backup/restore and rollback procedures are documented and tested.

Common failures

Symptom	Likely cause	Resolution
CORS error	Frontend domain absent/mismatched.	Set exact domain in `CORS_ORIGIN`, then redeploy backend.
API calls target localhost	Vite build used wrong/missing API URL.	Set `VITE_API_URL` in Vercel and rebuild.
Empty production data	Backend points to a different/empty database.	Check `MONGO_URI`; migrate/seed deliberately.
EdifyEight unavailable	Local upstream URL, token mismatch, or upstream down.	Use deployed HTTPS URLs; synchronize server-side tokens; check upstream health.
Uploads disappear/fail	Cloudinary not configured or local ephemeral storage used.	Set all Cloudinary variables and verify upload limits.
Inconsistent cache	Several instances using in-memory cache.	Provision shared Redis and confirm invalidation/namespace settings.
Nested route 404	SPA fallback missing or wrong root.	Confirm `vercel.json`, root directory and output folder.

Operational baseline

Monitor availability, response latency, error rate, database connectivity, Redis state, socket connections, external-service failures, authentication failures, privileged changes, storage usage and backup success. Use structured log aggregation and alert thresholds. Patch dependencies regularly and run role/access tests before every release.

8. Reusable blueprint for another project

Suggested implementation order

1. **Foundation:** define domains, user roles, permission vocabulary and ownership boundaries.
2. **Platform:** create frontend shell, backend structure, configuration validation, database connection, error format and logging.
3. **Identity:** implement users, password hashing, tokens/sessions, route guards and deny-by-default authorization.
4. **Core workflows:** build one complete vertical slice through route, controller, service, model, UI and tests.
5. **Shared services:** notifications, audit, uploads, reporting, cache and real-time communication.
6. **Integrations:** introduce backend-only adapters with timeouts, token handling, authorization and audit.
7. **Operations:** health/readiness endpoints, deployment manifests, backups, observability and rollback.

Recommended reusable structure

```
project/
├─ frontend/
│  └─ src/{app,api,components,features,hooks,layouts,routes,services,utils}
├─ backend/
│  ├─ config/ controllers/ routes/ services/ models/
│  ├─ middlewares/ modules/ infrastructure/ sockets/ logger/ jobs/
│  └─ scripts/ __tests__/
├─ docs/
├─ render.yaml
└─ README.md
```

Decisions to improve in the next project

- Define a single canonical frontend routing configuration to avoid duplicated menu/navigation sources.
- Use domain modules consistently across all backend features.
- Generate and version an OpenAPI specification from route schemas.
- Add integration/E2E tests for authentication, every role boundary, uploads, sockets and external adapters.
- Add CI gates for lint, tests, dependency scanning and build.
- Use migrations/controlled seeds, explicit API versioning, idempotency for sensitive writes, and standardized pagination/errors.
- Separate readiness from liveness checks and include dependency health without leaking internals.
- Use a secrets manager, short-lived credentials where possible, CSP, cookie strategy review, and formal retention policies.

> **Definition of done for the next project:** a feature is complete only when authorization, validation, audit, loading/error states, tests, observability, documentation and deployment configuration are included—not only the screen and endpoint.

Repository evidence used

Frontend/backend package manifests; routing and role configuration; route/controller/service/model structure; cache, logging and socket infrastructure; environment examples; Render and Vercel manifests; deployment and architecture documents; tests, scripts, Swagger output and Postman collection. This document describes the inspected repository as of the preparation date.